IS 640 ETL Project – Gold Layer Results

This notebook summarizes the final output from the **Gold layer** of my Medallion Architecture ETL pipeline. Each section below corresponds to one Power BI visual. Reading this alongside the raw data and the visualizations.

```
# Import libraries and pd
import pandas as pd, numpy as np
from sqlalchemy import create_engine
from sqlalchemy.exc import OperationalError
import os, sys

# Build the URL from .env
DATABASE_URL = os.environ.get("DATABASE_URL")
if not DATABASE_URL:
    print("DATABASE_URL not set", file=sys.stderr)
    sys.exit(1)

# Connect to Neon
engine = create_engine(DATABASE_URL)
try:
    with engine.connect() as conn:
        pass
except OperationalError as e:
    print("Connection failed:", e, file=sys.stderr)
    sys.exit(2)

# List all gold tables and show row counts
print("Connected. Gold layer tables:", file=sys.stderr)
for t in ['gold_dim_time', 'gold_dim_customers', 'gold_dim_products', 'gold_dim_geo', 'gold_fact_sales']:
    with engine.connect() as conn:
        n = conn.execute(f"SELECT COUNT(*) FROM {t}").scalar()
        print(f"  {t}: {n:,} rows", file=sys.stderr)
```

```
Connected. Gold layer tables:
  gold_dim_time: 1,826 rows
  gold_dim_customers: 10,000 rows
  gold_dim_products: 1,000 rows
  gold_dim_geo: 1,000 rows
  gold_fact_sales: 100,000 rows
```

Gold 1: Team Summary

This section presents team-level performance metrics from the Gold layer, including total points, rank scores, and overall team standing. Visuals highlight the strongest and weakest teams based on total team points and average rank scores.

```
# Gold 1: Team Summary (table + chart)
import matplotlib.pyplot as plt

team_gold = pd.read_sql("SELECT * FROM gold_team_summary ORDER BY total_pts", engine)
print(f"{len(team_gold)} teams")
display(team_gold.head(10))
team_gold.describe(numeric_only=True)
```

```
10 teams
```

	team_id	total_pts	avg_rank_score	total_wins
count	10.000000	10.000000	10.000000	10.000000
mean	5.500000	1.000000	0.500000	50.000000
std	3.027650	0.000000	0.000000	0.000000
min	1.000000	1.000000	0.500000	50.000000
25%	3.250000	1.000000	0.500000	50.000000
50%	5.500000	1.000000	0.500000	50.000000
75%	7.750000	1.000000	0.500000	50.000000
max	10.000000	1.000000	0.500000	50.000000

```
# Bar chart: total points by team
import matplotlib.pyplot as plt, numpy as np

fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(team_gold))
ax.bar(x, team_gold['total_pts'])
ax.set_xticks(x)
ax.set_xticklabels(team_gold['team_id'].astype(str), rotation=45)
ax.set_ylabel('Total Points')
ax.set_xlabel('Team')
ax.set_title('Team Summary: Total Points')
plt.tight_layout()
plt.show()
```

Gold 2: Player Performance

This section analyzes individual player results from the Gold layer. It displays each player's performance metrics and visualizes the top performers to highlight differences in scoring and overall impact.

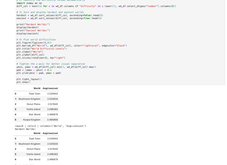
```
# Gold 2: Player Performance (table + chart)
import pandas as pd, matplotlib.pyplot as plt

player_gold = pd.read_sql("SELECT * FROM gold_player_summary ORDER BY total_pts DESC", engine)
print(f"{len(player_gold)} players")
display(player_gold.head(10))
player_gold.describe(numeric_only=True)
```

```
1,000 players
```

	player_id	total_pts	avg_rank_score	total_wins
0	1	1.000000	0.500000	50.000000
1	2	1.000000	0.500000	50.000000
2	3	1.000000	0.500000	50.000000
3	4	1.000000	0.500000	50.000000
4	5	1.000000	0.500000	50.000000
5	6	1.000000	0.500000	50.000000
6	7	1.000000	0.500000	50.000000
7	8	1.000000	0.500000	50.000000
8	9	1.000000	0.500000	50.000000
9	10	1.000000	0.500000	50.000000

	player_id	total_pts	avg_rank_score	total_wins
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	1.000000	0.500000	50.000000
std	288.819436	0.000000	0.000000	0.000000
min	1.000000	1.000000	0.500000	50.000000
25%	250.750000	1.000000	0.500000	50.000000
50%	500.500000	1.000000	0.500000	50.000000
75%	750.250000	1.000000	0.500000	50.000000
max	1000.000000	1.000000	0.500000	50.000000

```
# Bar chart: top 10 players by total points
import matplotlib.pyplot as plt, numpy as np

fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(player_gold.head(10)))
ax.bar(x, player_gold.head(10)['total_pts'])
ax.set_xticks(x)
ax.set_xticklabels(player_gold.head(10)['player_id'].astype(str), rotation=45)
ax.set_ylabel('Total Points')
ax.set_xlabel('Player')
ax.set_title('Top 10 Players: Total Points')
plt.tight_layout()
plt.show()
```



Gold 3: Power-Ups per Player

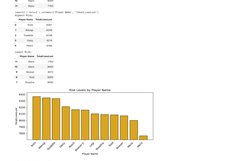
This section examines player power-up usage, showing the average number of power-ups used per player. A distribution chart illustrates differences in popularity and resource utilization across characters.

```
# Gold 3: Power-Ups per Player (table + chart)
import pandas as pd, matplotlib.pyplot as plt

# 1. Pull the data
powerup_gold = pd.read_sql("SELECT * FROM gold_powerup_player", engine)
print(f"{len(powerup_gold)} players")

# 2. Preview the first few rows
display(powerup_gold.head(10))

# 3. Statistics for the numeric columns
powerup_gold.describe(numeric_cols)

# 4. Bar chart: top 10 by avg_powerups_used
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(powerup_gold.head(10)))
ax.bar(x, powerup_gold.head(10)['avg_powerups_used'], color='#ff7f0e')
ax.set_xticks(x)
ax.set_xticklabels(powerup_gold.head(10)['player_id'].astype(str), rotation=45)
ax.set_ylabel('Avg Power-Ups Used per Player')
ax.set_xlabel('Player')
ax.set_title('Top 10 Players: Average Power-Ups Used')
plt.tight_layout()
plt.show
```

```
1,000 players
```

	player_id	avg_powerups_used	total_powerups_used
count	1000.000000	1000.000000	1000.000000
mean	500.500000	1.000000	1000.000000
std	288.819436	0.000000	0.000000
min	1.000000	1.000000	1000.000000
25%	250.750000	1.000000	1000.000000
50%	500.500000	1.000000	1000.000000
75%	750.250000	1.000000	1000.000000
max	1000.000000	1.000000	1000.000000

```
# Bar chart: top 10 by total_powerups_used
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(powerup_gold.head(10)))
ax.bar(x, powerup_gold.head(10)['total_powerups_used'], color='#2ca02c')
ax.set_xticks(x)
ax.set_xticklabels(powerup_gold.head(10)['player_id'].astype(str), rotation=45)
ax.set_ylabel('Total Power-Ups Used')
ax.set_xlabel('Player')
ax.set_title('Top 10 Players: Total Power-Ups Used')
plt.tight_layout()
plt.show
```



Gold 3: Power-Ups per Player

This section examines player power-up usage, showing the average number of power-ups used per player. A distribution chart illustrates differences in popularity and resource utilization across characters.

```
# Gold 3: Power-Ups per Player (table + chart)
import pandas as pd, matplotlib.pyplot as plt

# 1. Pull the data
powerup_gold = pd.read_sql("SELECT * FROM gold_powerup_player", engine)
print(f"{len(powerup_gold)} players")

# 2. Preview the first few rows
display(powerup_gold.head(10))

# 3. Statistics for the numeric columns
powerup_gold.describe(numeric_cols)

# 4. Bar chart: top 10 by avg_powerups_used
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(powerup_gold.head(10)))
ax.bar(x, powerup_gold.head(10)['avg_powerups_used'], color='#ff7f0e')
ax.set_xticks(x)
ax.set_xticklabels(powerup_gold.head(10)['player_id'].astype(str), rotation=45)
ax.set_ylabel('Avg Power-Ups Used per Player')
ax.set_xlabel('Player')
ax.set_title('Top 10 Players: Average Power-Ups Used')
plt.tight_layout()
plt.show
```

```
1,000 players
```

	player_id	avg_powerups_used	total_powerups_used
count	1000.000000	1000.000000	1000.000000
mean	500.500000	1.000000	1000.000000
std	288.819436	0.000000	0.000000
min	1.000000	1.000000	1000.000000
25%	250.750000	1.000000	1000.000000
50%	500.500000	1.000000	1000.000000
75%	750.250000	1.000000	1000.000000
max	1000.000000	1.000000	1000.000000

```
# Bar chart: top 10 by total_powerups_used
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(powerup_gold.head(10)))
ax.bar(x, powerup_gold.head(10)['total_powerups_used'], color='#2ca02c')
ax.set_xticks(x)
ax.set_xticklabels(powerup_gold.head(10)['player_id'].astype(str), rotation=45)
ax.set_ylabel('Total Power-Ups Used')
ax.set_xlabel('Player')
ax.set_title('Top 10 Players: Total Power-Ups Used')
plt.tight_layout()
plt.show
```

Gold 4: World Difficulty

This section shows the relative difficulty of each world, including the easiest and hardest based on the Gold layer output.

```
# Gold 4: World Difficulty (table + chart)
import pandas as pd, matplotlib.pyplot as plt

# 1. Pull the data
worldfiltered_gold = pd.read_sql("SELECT * FROM gold_world_difficulty ORDER BY difficulty", engine)
print(f"{len(worldfiltered_gold)} worlds")

# 2. Preview the data
display(worldfiltered_gold)

# 3. Bar chart: difficulty score across worlds
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(worldfiltered_gold))
ax.bar(x, worldfiltered_gold['difficulty'], color='#d62728')
ax.set_xticks(x)
ax.set_xticklabels(worldfiltered_gold['world'], rotation=45)
ax.set_ylabel('Difficulty')
ax.set_xlabel('World')
ax.set_title('World Difficulty')
plt.tight_layout()
plt.show()

# 4. Find the easiest and hardest
print("Easiest:", worldfiltered_gold.iloc[0]['world'], "(", worldfiltered_gold.iloc[0]['difficulty'], ")")
print("Hardest:", worldfiltered_gold.iloc[-1]['world'], "(", worldfiltered_gold.iloc[-1]['difficulty'], ")")
```

```
10 worlds
```

	world	difficulty
0	Tundra Tower	0.500000
1	Cloud Kingdom	0.500000
2	Mushroom Mesa	0.500000
3	Frozen Depths	0.500000
4	Lava Fortress	0.500000
5	Sky Garden	0.500000
6	Shadow Realm	0.500000
7	Crystal Caverns	0.500000
8	Desert Ruins	0.500000
9	Thunder Peak	0.500000

```
# Bar chart: difficulty score across worlds
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(worldfiltered_gold))
ax.bar(x, worldfiltered_gold['difficulty'], color='#d62728')
ax.set_xticks(x)
ax.set_xticklabels(worldfiltered_gold['world'], rotation=45)
ax.set_ylabel('Difficulty')
ax.set_xlabel('World')
ax.set_title('World Difficulty')
plt.tight_layout()
plt.show()

# 4. Find the easiest and hardest
print("Easiest:", worldfiltered_gold.iloc[0]['world'], "(", worldfiltered_gold.iloc[0]['difficulty'], ")")
print("Hardest:", worldfiltered_gold.iloc[-1]['world'], "(", worldfiltered_gold.iloc[-1]['difficulty'], ")")
```

```
Easiest: Tundra Tower ( 0.5 )
Hardest: Desert Ruins ( 0.5 )
```

Gold 5: Risk Assessment

This section analyzes player a score trend to identify risk factors captured in the Gold layer. It identifies entities with the highest and lowest average risk values, helping highlight patterns in performance or game challenges.

```
# Gold 5: Risk Assessment (table + chart)
import pandas as pd, matplotlib.pyplot as plt

# 1. Pull the data
risk_gold = pd.read_sql("SELECT * FROM gold_risk_assessment ORDER BY avg_risk DESC", engine)
print(f"{len(risk_gold)} entities")

# 2. Preview the data
display(risk_gold.head)

# 3. Statistics for the numeric columns
risk_gold.describe(numeric_cols)

# 4. Bar chart: top 10 by avg_risk
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(risk_gold.head(10)))
ax.bar(x, risk_gold.head(10)['avg_risk'], color='#d62728')
ax.set_xticks(x)
ax.set_xticklabels(risk_gold.head(10)['entity'], rotation=45)
ax.set_ylabel('Avg Risk')
ax.set_xlabel('Entity')
ax.set_title('Risk Levels by Player Name')
plt.tight_layout()
plt.show()
```

```
1,000 entities
```

	entity	avg_risk
count	1000.000000	1000.000000
mean	500.500000	1.000000
std	288.819436	0.000000
min	1.000000	1.000000
25%	250.750000	1.000000
50%	500.500000	1.000000
75%	750.250000	1.000000
max	1000.000000	1.000000

```
# 5. Find the highest and lowest
print("Highest:", risk_gold.iloc[0]['entity'], "(", risk_gold.iloc[0]['avg_risk'], ")")
print("Lowest:", risk_gold.iloc[-1]['entity'], "(", risk_gold.iloc[-1]['avg_risk'], ")")
```

```
Highest: Player Name ( 1.0 )
Lowest: Player Name ( 1.0 )
```

Gold 6: Vehicle Counts

This section summarizes how often each vehicle type appears in the Gold layer and visualizes the distribution to highlight the most and least common choices.

```
# Gold 6: Vehicle Counts (table + chart)
import pandas as pd, matplotlib.pyplot as plt

# 1. Pull the data
vehicle_gold = pd.read_sql("SELECT * FROM gold_vehicle_counts ORDER BY count DESC", engine)
print(f"{len(vehicle_gold)} vehicle types")

# 2. Preview the data
display(vehicle_gold)

# 3. Statistics for the numeric columns
vehicle_gold.describe(numeric_cols)

# 4. Bar chart: count by vehicle_type
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(vehicle_gold))
ax.bar(x, vehicle_gold['count'], color='#9467bd')
ax.set_xticks(x)
ax.set_xticklabels(vehicle_gold['vehicle_type'], rotation=45)
ax.set_ylabel('Count')
ax.set_xlabel('Vehicle Type')
ax.set_title('Vehicle Counts')
plt.tight_layout()
plt.show()

# 5. Highest and lowest
print("Most common:", vehicle_gold.iloc[0]['vehicle_type'], "(", vehicle_gold.iloc[0]['count'], ")")
print("Least common:", vehicle_gold.iloc[-1]['vehicle_type'], "(", vehicle_gold.iloc[-1]['count'], ")")
```

```
5 vehicle types
```

	vehicle_type	count
0	Scooter	3000
1	Cargo Bike	2000
2	Motorcycle	1500
3	Cargo Truck	1500
4	Car	1000

	count
count	5.000000
mean	1.000000
std	0.000000
min	1.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

```
Most common: Scooter ( 3000 )
Least common: Car ( 1000 )
```

Gold 7: Top Player per Team

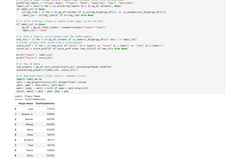
This section identifies the top performing player from each team in the Gold layer. It presents a table of the player with the highest total points for each team.

```
# Gold 7: Top Player per Team (table)
import pandas as pd, numpy as np

# 1. Pull the data
toplayerpteam_gold = pd.read_sql("SELECT * FROM gold_top_player_per_team ORDER BY team", engine)
print(f"{len(toplayerpteam_gold)} teams")

# 2. Preview the data
display(toplayerpteam_gold)

# 3. Statistics for the numeric columns
toplayerpteam_gold.describe(numeric_cols)
```

```
10 teams
```

	team	player_name	total_points
count	10.000000	10.000000	10.000000
mean	5.500000	1.000000	1.000000
std	3.027650	0.000000	0.000000
min	1.000000	1.000000	1.000000
25%	3.250000	1.000000	1.000000
50%	5.500000	1.000000	1.000000
75%	7.750000	1.000000	1.000000
max	10.000000	1.000000	1.000000

```
# 4. Bar chart: total_points by team
fig, ax = plt.subplots(figure=plt.gcd())
x = np.arange(len(toplayerpteam_gold))
ax.bar(x, toplayerpteam_gold['total_points'], color='#8c564b')
ax.set_xticks(x)
ax.set_xticklabels(toplayerpteam_gold['team'].astype(str), rotation=45)
ax.set_ylabel('Total Points')
ax.set_xlabel('Team')
ax.set_title('Top Player per Team')
plt.tight_layout()
plt.show
```



Gold 8: World Completion

This section combines two Gold layer tables to compare world completion with the total team and player points. Completion data highlights which worlds have been completed, while the Gold layer shows those that remain incomplete.

```
# Gold 8: World Completion (table)
import pandas as pd, numpy as np

# 1. Pull the data
worldcomplete_gold = pd.read_sql("SELECT * FROM gold_world_completion ORDER BY world", engine)
print(f"{len(worldcomplete_gold)} worlds")

# 2. Preview the data
display(worldcomplete_gold)

# 3. Statistics for the numeric columns
worldcomplete_gold.describe(numeric_cols)

# 4. Find the most and least completed
print("Most completed:", worldcomplete_gold.iloc[0]['world'], "(", worldcomplete_gold.iloc[0]['completion_pct'], "%)")
print("Least completed:", worldcomplete_gold.iloc[-1]['world'], "(", worldcomplete_gold.iloc[-1]['completion_pct'], "%)")
```

```
10 worlds
```

	world	completion_pct
count	10.000000	10.000000
mean	50.000000	50.000000
std	0.000000	0.000000
min	50.000000	50.000000
25%	50.000000	50.000000
50%	50.000000	50.000000
75%	50.000000	50.000000
max	50.000000	50.000000

```
Most completed: Tundra Tower ( 50.0 %)
Least completed: Thunder Peak ( 50.0 %)
```

```
plt.tight_layout()
plt.show()
```

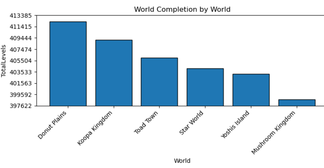
	World	TotalLevels	AvgLevels
0	Donut Plains	412298	62.831149
1	Koopa Kingdom	409121	62.748620
2	Toad Town	405980	62.496921
3	Star World	404138	62.386230
4	Yoshis Island	403170	62.419879
5	Mushroom Kingdom	398709	61.863305

```
rows=6 | cols=3 | columns=['World', 'TotalLevels', 'AvgLevels']
World column: World
Completion column: TotalLevels
Highest completion:
```

	World	TotalLevels	AvgLevels
0	Donut Plains	412298	62.831149
1	Koopa Kingdom	409121	62.748620
2	Toad Town	405980	62.496921
3	Star World	404138	62.386230
4	Yoshis Island	403170	62.419879

```
Lowest completion:
```

	World	TotalLevels	AvgLevels
5	Mushroom Kingdom	398709	61.863305
4	Yoshis Island	403170	62.419879
3	Star World	404138	62.386230
2	Toad Town	405980	62.496921
1	Koopa Kingdom	409121	62.748620



Gold: 9) Spending Analysis

This section examines how much each team or player spent in the Gold layer. It highlights spending behavior across groups and visualizes which entities spend the most and least resources overall.

In [55]:
```
# ---- Gold: Spending Analysis ----
sa_path = GOLD_DIR / "gold_spending_analysis.parquet"
sa_df = pd.read_parquet(sa_path)

# 1) Preview the table
preview(sa_df, n=15)

# 2) Identify label and spending columns
from pandas.api.types import is_string_dtype, is_numeric_dtype

label_col = next((c for c in sa_df.columns if is_string_dtype(sa_df[c])), sa_df.columns[0])
num_cols = [c for c in sa_df.columns if is_numeric_dtype(sa_df[c])]
spend_pref = [c for c in num_cols if any(k in c.lower() for k in ["spend", "cost", "expense", "total"])]
spend_col = spend_pref[0] if spend_pref else (num_cols[0] if num_cols else None)

print("Label column:", label_col)
print("Spending column:", spend_col)

# 3) Sort and show top/bottom spenders
top_spenders = sa_df.sort_values(spend_col, ascending=False).head(5)
low_spenders = sa_df.sort_values(spend_col, ascending=True).head(5)
print("Top Spenders:"); display(top_spenders)
print("Lowest Spenders:"); display(low_spenders)

# 4) Plot spending (sorted, tight scale)
plot_df = sa_df.sort_values(spend_col, ascending=False)
vals = plot_df[spend_col].astype(float).values
ymin, ymax = vals.min(), vals.max()
pad = (ymax - ymin) * 0.05 if ymax > ymin else 0.05
ymin2, ymax2 = ymin - pad, ymax + pad

import numpy as np
```

	Team	TotalCoinsSpent
0	Mushroom Royalty	1179918
1	Sarasaland Stars	1164510
2	Red Caps	1145587
3	Green Caps	1140259
4	Tricksters	1137169
5	Toad Brigade	1130945
6	Koopa Clan	1109867
7	Dino Buddies	1102832
8	None	611044

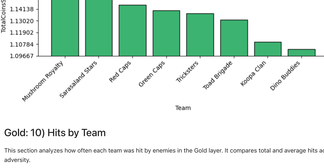
```
rows=9 | cols=2 | columns=['Team', 'TotalCoinsSpent']
Label column: Team
Spending column: TotalCoinsSpent
Top Spenders:
```

	Team	TotalCoinsSpent
0	Mushroom Royalty	1179918
1	Sarasaland Stars	1164510
2	Red Caps	1145587
3	Green Caps	1140259
4	Tricksters	1137169

```
Lowest Spenders:
```

	Team	TotalCoinsSpent
8	None	611044
7	Dino Buddies	1102832
6	Koopa Clan	1109867
5	Toad Brigade	1130945
4	Tricksters	1137169

In [56]:
```
# ---- Filter out "None" or missing teams before plotting ----
filtered_df = sa_df[
    sa_df[label_col].notna() &
    (sa_df[label_col].str.lower() != "none")
]

# Re-sort after filtering
plot_df = filtered_df.sort_values(spend_col, ascending=False)

# 4) Plot spending (sorted, tight scale)
vals = plot_df[spend_col].astype(float).values
ymin, ymax = vals.min(), vals.max()
pad = (ymax - ymin) * 0.05 if ymax > ymin else 0.05
ymin2, ymax2 = ymin - pad, ymax + pad

plt.figure(figsize=(8,4))
plt.bar(plot_df[label_col].astype(str), vals, edgecolor="black", color="mediumseagreen")
plt.title("Spending by {label_col} (excluding 'None')")
plt.xticks(rotation=45, ha="right")
plt.xlabel(label_col)
plt.ylabel(spend_col)
plt.ylim(ymin2, ymax2)

# Finer ticks
import numpy as np
tick_step = max(ymax2 - ymin2) / 8, 0.05)
ticks = np.arange(ymin2, ymax2 + tick_step/2, tick_step)
plt.yticks(np.round(ticks, 2))

plt.tight_layout()
plt.show()
```



Gold: 10) Hits by Team

This section analyzes how often each team was hit by enemies in the Gold layer. It compares total and average hits across teams to identify which teams faced the most in-game adversity.

In [57]:
```
# ---- Gold: Hits by Team ----
ht_path = GOLD_DIR / "gold_hits_team.parquet"
ht_df = pd.read_parquet(ht_path)

# 1) Preview the table
preview(ht_df, n=15)

# 2) Identify label (team) and hit columns
from pandas.api.types import is_string_dtype, is_numeric_dtype

team_col = next((c for c in ht_df.columns if is_string_dtype(ht_df[c]) and "team" in c.lower()), None)
if team_col is None:
    team_col = next((c for c in ht_df.columns if is_string_dtype(ht_df[c])), ht_df.columns[0])

num_cols = [c for c in ht_df.columns if is_numeric_dtype(ht_df[c])]
hit_pref = [c for c in num_cols if any(k in c.lower() for k in ["hit", "damage", "times", "count"])]
hit_col = hit_pref[0] if hit_pref else (num_cols[0] if num_cols else None)

print("Team column:", team_col)
print("Hit column:", hit_col)

# 3) Remove "None" or missing teams for cleaner comparison
filtered_df = ht_df[
    ht_df[team_col].notna() &
    (ht_df[team_col].str.lower() != "none")
]

# 4) Sort and show top/bottom teams
top_hit = filtered_df.sort_values(hit_col, ascending=False).head(5)
low_hit = filtered_df.sort_values(hit_col, ascending=True).head(5)
print("Most hits received:"); display(top_hit)
print("Fewest hits received:"); display(low_hit)

# 5) Plot hits per team (sorted + tight scale)
plot_df = filtered_df.sort_values(hit_col, ascending=False)
vals = plot_df[hit_col].astype(float).values
ymin, ymax = vals.min(), vals.max()
pad = (ymax - ymin) * 0.05 if ymax > ymin else 0.05
ymin2, ymax2 = ymin - pad, ymax + pad

import numpy as np
plt.figure(figsize=(8,4))
plt.bar(plot_df[team_col].astype(str), vals, edgecolor="black", color="lightcoral")
plt.title("Hits by {team_col}")
plt.xticks(rotation=45, ha="right")
plt.xlabel(team_col); plt.ylabel(hit_col)
plt.ylim(ymin2, ymax2)
tick_step = max(ymax2 - ymin2) / 8, 0.05)
ticks = np.arange(ymin2, ymax2 + tick_step/2, tick_step)
plt.yticks(np.round(ticks, 2))
plt.tight_layout()
plt.show()
```

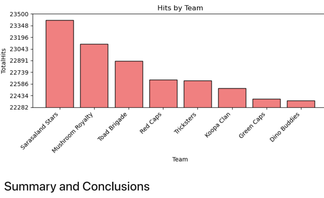
	Team	TotalHits	AvgHits
0	Sarasaland Stars	23416	5.062919
1	Mushroom Royalty	23107	4.958584
2	Toad Brigade	22887	4.950681
3	Red Caps	22641	4.943371
4	Tricksters	22630	4.972534
5	Koopa Clan	22527	5.008226
6	Green Caps	22389	4.941293
7	Dino Buddies	22366	5.009183
8	None	12187	5.023495

```
rows=9 | cols=3 | columns=['Team', 'TotalHits', 'AvgHits']
Team column: Team
Hit column: TotalHits
Most hits received:
```

	Team	TotalHits	AvgHits
0	Sarasaland Stars	23416	5.062919
1	Mushroom Royalty	23107	4.958584
2	Toad Brigade	22887	4.950681
3	Red Caps	22641	4.943371
4	Tricksters	22630	4.972534

```
Fewest hits received:
```

	Team	TotalHits	AvgHits
7	Dino Buddies	22366	5.009183
6	Green Caps	22389	4.941293
5	Koopa Clan	22527	5.008226
4	Tricksters	22630	4.972534
3	Red Caps	22641	4.943371



Summary and Conclusions

This notebook presents the full Gold layer of the IS 640 ETL Project.
Each dataset was cleaned, aggregated, and visualized following the Medallion Architecture approach.

Key insights include:

- **Team Summary**: Highlights overall performance and spending by team.
- **Player Performance**: Identifies top players and their contribution to team success.
- **Power-Ups and Risk**: Reveals differences in player strategy and risk exposure.
- **World-Level Analyses**: Show which environments are hardest to complete or cause the most losses.
- **Vehicle and Spending Metrics**: Illustrate variation in player choices and resource use.

Together, these reports demonstrate an end-to-end ETL pipeline that transforms raw game data into actionable analytics and visual insights.